

Project 1 Report

Scraping Event Information from Visit Stockholm

Digital Strategies for the Social Sciences · Linköping University · Patricia Givort Cruz Cabral

1. Guiding Question

I sent my R script to Hadeer Ali Khalifa Ali and I ran the script of Hadeer Ali Khalifa Ali.

My guiding question for this project is: What kinds of events are currently promoted on the Visit Stockholm events page, and how are they distributed by category? I chose this page because it contains public information about cultural events in Stockholm, like event names, categories, dates, locations, and links to individual event pages.

Since I live in Sweden, I also found this personally interesting. As a foreign student, I often miss events in Stockholm because I do not check event websites regularly. I saw this as a first step toward building a structured event list that could connect to a personal calendar, making it easier to follow what is happening without checking the website all the time.

Ideally, I would have liked to collect events for the whole year, but the website's calendar interface depends on dynamic date filters that require browser automation to access. The dataset should be understood as a snapshot of the events available at the time of scraping.

For this project, I worked only with public event information from an official tourism website. I did not collect personal data or use login credentials, and I added pauses between all requests. The data collected relates exclusively to public cultural events and contains no sensitive or personal information.

2. Downloading and Parsing the Data

The first step was to download the main events page using GET() and check the HTTP status code. The request returned status code 200, which meant the page was downloaded successfully. I also checked the response headers and confirmed that the content type was HTML with UTF-8 encoding. After that, I converted the raw response content into readable HTML text and saved a local copy of the page as visitstockholm.html..

I parsed the HTML into a DOM tree using htmlParse(). This allowed me to use XPath to explore the structure of the page. I tested different tags such as h1, h2, h3, a, p, span, div, and time. The event names were stored in h3 elements, and the event links were stored in a elements. I used a regular expression to keep only links that followed the Visit Stockholm event page URL pattern.

3. Difficulties

One difficulty I had was that the event cards were not structured as a clean HTML table. The information I needed, like dates, was mixed with other page elements. At first, I tried to extract information from paragraph tags and time tags, but they did not contain the data I needed. I also tested div tags, but the result was too long and messy (more than 9999 lines of output) because the div elements contained many nested parts of the page. This showed me one limitation of web scraping and that HTML is often designed for how the page looks to users, not for easy data extraction.

To fix this, I extracted all text elements from the page and inspected their order. When I did that, I found a repeated pattern around the phrase “Calendar icon” I used this pattern to extract the event information based on the position of each element. Since this method depended on relative positions in the HTML, I added an extra cleaning step and created a list of valid categories based on the website filters, such as Music, Exhibitions... and kept only the rows where the extracted category matched one of these valid categories.

After analyzing the results, I also found that some events appeared more than once because the website displayed them in different sections. So I checked for duplicates using the event name and link together, and removed duplicated rows, this helped keep the final dataset as one row per unique event.

Another difficulty appeared when I visited the individual event subpages. Some pages listed only the venue, street address, and city, while others included a postal code between the street address and the city. In my first version, some postal codes were accidentally stored in the city column. To fix this, I added a separate postal code field and used a regular expression to detect Swedish postal codes. Figure 1 below shows an extract of the address columns from the dataset after the fix was applied.

K	L
reet_address	city
'enaslingan 14	121 77
'enatorget	121 77
'enatorget	121 77
'enatorget	121 77
tipelagstigen 1	134 40
tipelagstigen 1	134 40
inellvägen 8	114 28
unkebergstorg 2 - 4	111 51
unnsgatan 21A	11138
lamottevägen 5	13440
urgårdsslätten 43-45	115 21
urgårdsvägen 6-16	115 21
atenvägen 200	138 31
stav Adolfs torg 2	Stockholm
rnstulls strand 4	117 39
rnstulls strand	Stockholm

Figure 1. Extract of the street_address and city columns from the scraped dataset, showing the result after the postal code fix was applied. .

I also had a small encoding issue when opening the saved CSV file in Excel. Some Swedish characters, such as å, ä, and ö, and some dashes appeared as strange symbols. I had to do some research to understand what was happening. In R, I saw that the website used UTF-8 encoding, so the issue seemed to come from how Excel interprets CSV files by default. To reduce this problem, I used write_excel_csv() from the readr package, which works better when opening CSV files with special characters in Excel. Figure 2 shows some of these cases.

Avicii Aren	Olivia Dean - The Art of Loving	!
Avicii Aren	NE-YO & AKON "Nights Like This Tou	!
Cirkus	& Juliet - The Musical	!
Drejeriet	Pottery wheel throwing	!
Flatenbast	Flatenbastun "Lakeside Sauna Exper	!

Figure 2. Example of the encoding issue found when opening the CSV in Excel. Swedish characters like å, ä, and ö appeared as strange symbols.

I also tried to use Selenium to collect more events across multiple pages. The Chrome attempt failed because the ChromeDriver version did not match my browser, and the Firefox attempt failed because RSelenium could not download the required dependency. Since this made the script less reproducible, I decided not to use Selenium in the final version and kept the project to static HTML scraping.

```
> rD <- rsDriver(
-   browser = "firefox",
-   verbose = FALSE
- )

Error in open.connection(con, "rb") :
  cannot open the connection to 'https://api.bitbucket.org/2.0/repositories/ariya/phantomjs/downloads?pag
e=1&size=20'

In addition:
Warning message:
In open.connection(con, "rb") :
  cannot open URL 'https://api.bitbucket.org/2.0/repositories/ariya/phantomjs/downloads?pag
e=1&size=20': HTTP status=403 Forbidden
Required'
```

Figure .3 Error while trying to use selenium

4. Visiting Multiple Pages

I used the event links extracted from the main page to automatically visit each individual event subpage in a “for-loop.” From these subpages, I extracted the event title, status code, description, venue name, street address, postal code, and city. This made the dataset richer.

I also downloaded one event image from the Open Graph metadata of the first event subpage and saved it locally. A one-second pause was added between all requests using `Sys.sleep(1)`.

5. Analysis and Visualisation

For the analysis, I created summary tables of events by category, location, and duration type. Table 1 below shows a sample of the cleaned dataset with the main variables extracted, and Table 2 shows how events are split between single-day and multi-day.

Sample of the scraped events dataset			
event_name	category	date	location
The Hornstull Market 2026	Fairs	May 16 - Sep 27	Hornstulls Strand
Folk music and dance evenings	Music	May 17	Skeppsholmens Folkhögskola/Föreningen Folkmusikhuset
The Viking World – the world’s largest Viking exhibition	Exhibitions	May 13 - Jan 1	Swedish History Museum
Springbeast Festival 2026 – Urban Art & Music Festival in Stockholm	Festivals	May 14 - May 17	Kulturkvarter Snösätra
The Fredrik Roos Scholarship exhibition 2026: Kim Andreas Roland Berger – Accumulated Stacks	Exhibitions	May 13 - May 14	Artipelag
L’eraclito amoroso, listen All Lovers	Music	May 16	Panncentralen Gustavsberg
Quarnevalen 2026	Festivals	May 16	KTH
Exhibition Scapes, by Alia Ali	Exhibitions	May 13 - May 17	House of Photography
Sasha Velour	Stage & Film	May 13	Göta Lejon
Ulf Stureson at Debaser Nova	Music	May 13	Debaser Nova

Table 1. Sample of the cleaned events dataset showing the main variables extracted

I also created the `event_duration_type` variable because the original date column contained both single dates and date ranges. Table 2 below shows the number of events classified as single-day and multi-day events. This was useful for describing the type of events promoted on the website, and it also connects to the idea of a future calendar integration, where single-day and multi-day events would need to be handled differently.

Number of events by duration type	
event_duration_type	number_of_events
Multi-day event	13
Single-day event	14

Table 2. Number of events by duration type, showing how many events in the dataset span a single day versus multiple days.

I also looked at the location distribution, but since the dataset is a snapshot from around May, I focused the main visualization on categories, which were more informative.

As an extra step, I used the Nominatim API to add latitude and longitude to the dataset. I only sent unique queries, added a User-Agent header, and paused one second between requests.

I generate a bar chart like show in the Figure 4 below, with the distribution of events by category scraped from the Visit Stockholm events page. Music was the most represented category with 8 events, followed by Exhibitions with 6 and Sports & Wellbeing with 5. Categories such as Networking & Community and Fairs had fewer than 2 events each at the time of scraping. The horizontal layout was chosen because some category names are long and would have overlapped on a vertical axis.

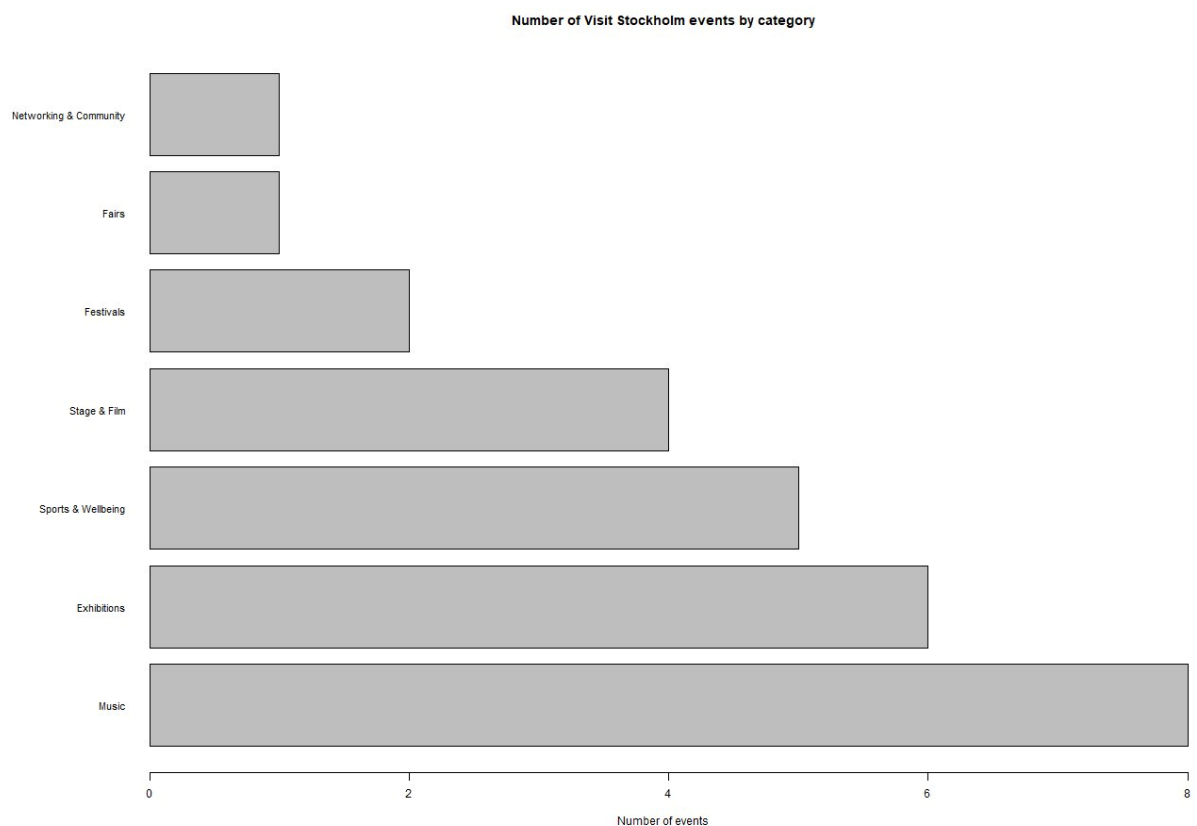


Figure 4. Number of events by category scraped from the Visit Stockholm events page (visitstockholm.com/events/). Music is the most common category, followed by Exhibitions and Sports & Wellbeing.

This project gave me a clearer picture of what kinds of events are promoted in Stockholm and how a scraper can be built step by step, from downloading a single page to visiting subpages and enriching the data with an API. The dataset is monthly because of the pagination limitation, but it works as a proof of concept for the calendar integration idea I had in mind from the beginning

6. Ethical and Legal Reflections

The data I collected is entirely public, the Visit Stockholm is an official tourism promotion site and the event information it displays is what the website is designed to share with anyone who visits it. There was no login, no paywall or personal data involved at any point. Because of this, the dataset does not fall under the scope of GDPR's personal data provisions.

I tried to be careful about how the requests were made, since the script visits each event subpage individually and also calls the Nominatim geocoding API for each unique address, I added a one-second pause using `System.sleep(1)` inside both loops. The Nominatim API explicitly asks for no more than one request per second in its usage policy. I also made sure to only send unique geocoding queries, so the same address would not be requested twice.

For the Nominatim API, I added a descriptive User-Agent header to every request, as their policy requires. This identifies the application so the API operators can reach out if there is a problem. I did not use any API key or create any account, and the geocoordinate data was only used locally for the dataset.

The images downloaded from each event page came from the Open Graph metadata, which websites publish specifically so that link previews can be generated and these images were stored locally and used only for inspection, they were not republished or shared, I tried to keep the data collection as narrow down as possible only what was needed to answer the research question and to interact with the website and APIs in a way that respects their infrastructure and their terms of use.